Library System Assignment

C24372973 Mohammed Younis

1. What the Website Does

This is a library website I built. Users can make an account, log in, search for books, reserve books, and manage their reservations. I made this website by looking at your notes and adapting it to my assignment, I took my time to understand your code and use it for the assignment. For the styling I looked at a couple of library websites (e.g Chicago library webstite) and took inspiration from them.

I am explaining how each page works very thoroughly in the 4th section

2. Files I Created

database.php - Connects to the database. **dashboard.php** - The homepage. **login.php** - The login page. **register.php** - The sign up page.

dashboard.php - The main page after logging in. **logout.php** - Logs you out. **style.css** - All the styling and colors.

3. How It Works

You start at the login page. If you don't have an account, you can register.

After logging in, you go to the dashboard. Here you can search for books by title, author, or category.

If a book is available, you can reserve it. If it's already reserved, you can't.

You can see all books you've reserved and cancel them if you want.

4. Code Examples and explaining exactly what I did

- Here's how the login and logout works:

The login page first of all starts a session and then connects to the database then has an if statement to check if a form was submitted and if it is, it is dealt with by checking if the user is registered and inputted his login details correctly as shown below.

```
$result = $conn->query("SELECT * FROM Users WHERE Username='$user' AND
Password='$pass'");
if ($result->num_rows > 0) {
    $_SESSION['username'] = $user;
    header("Location: dashboard.php");
}
} else {
        $error = "Invalid login credentials";
    }
}
```

The code above uses an sql query to check that the username and password are in the database and allows the user to proceed or give him a an error and forward him back to the same page where they can then register and once they register it will log them in and let them proceed to the next page where they can find books and reserve them if they aren't already reserved.

Below that in my code I have the forms which use POST to send data to the server, I set the input type to be “password” so that the information is hidden which enhances the data’s security as people can't just look over at their password.

- For the logout page, it just destroys the session

- Here is what the **registration** page does:

Checks if the form was submitted using “isset” and then takes the user the everything the user inputted and makes it safe to insert into a database by using “real_escape_string” which deals with any special character inputted with the name like “O’Neills” and then inserts it into the

database, it also check if the mobile number the user inputted is exactly 10 digits and the code also checks if the user is already in the database and won't allow them to use someone else's database as the username is the primary key in the database.

Below that code is the form which is very similar to the login form as I wanted to keep it consistent.

Sample code:

```
// Handle registration and checks registration form was submitted
if (isset($_POST['register'])) {
    $user = $conn->real_escape_string($_POST['username']);
    $pass = $conn->real_escape_string($_POST['password']);
    $first = $conn->real_escape_string($_POST['firstname']);
    $last = $conn->real_escape_string($_POST['surname']);
    $mobile = $conn->real_escape_string($_POST['mobile']);

    if (strlen($mobile) != 10 || !is_numeric($mobile)) {
        $error = "Mobile number must be exactly 10 digits";
    } else {
        $check = $conn->query("SELECT * FROM Users WHERE
Username='$user'");
        if ($check->num_rows == 0) { //checks if the username already
exists by checking if there are any rows with that username
            $conn->query("INSERT INTO Users (Username, Password,
FirstName, Surname, Mobile)
                        VALUES ('$user', '$pass', '$first', '$last',
'$mobile')");
            $_SESSION['username'] = $user; //logs the user
automatically after they register
            header("Location: dashboard.php");
```

```
            exit;
        } else {
            $error = "Username already exists";
        }
    }
}
```

- I also have a page for the **database**, this page connects to the database and will send an error message if the connect fails as a form of error handling.

- The dashboard page, the one that does the search, reservation etc..:

  This page composes of a lot of if statements to handle each form like the search function which I will explain:

  The way it works is by checking if the user submitted the search form then uses an sql query to select all 6 columns in book and then do a left join to get the book category as well.

  Its all the same logic of using if statements to check is a form is submitted and if it is an sql query will insert, delete, update etc.

  For the forms they also follow the same logic, It has buttons and sql queries to show the books avaible to search for a specific one,

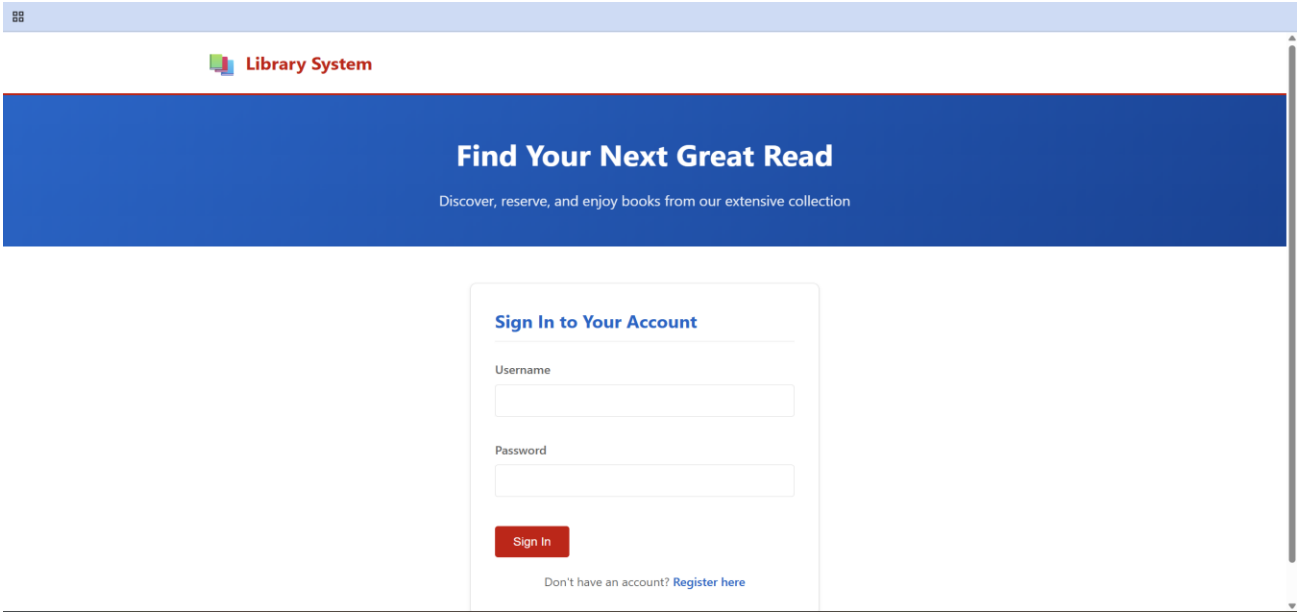
Sample code:

```
$books = [];

if (isset($_GET['search'])) {

    $title = $conn->real_escape_string($_GET['title'] ?? ''); //
used ?? to provide a default value if the parameter is not set

    $author = $conn->real_escape_string($_GET['author'] ?? '');

    $category = $conn->real_escape_string($_GET['category'] ??
'');




    $sql = "SELECT b.*, c.CategoryDescription FROM Books b

            LEFT JOIN Categories c ON b.CategoryID = c.CategoryID
";



    if (!empty($title)) $sql .= " AND b.BookTitle LIKE
'%$title%'";

    if (!empty($author)) $sql .= " AND b.Author LIKE
'%$author%'";

    if (!empty($category)) $sql .= " AND b.CategoryID =
'$category'";



    $result = $conn->query($sql);

    if ($result) {

        $books = $result->fetch_all(MYSQLI_ASSOC);
```

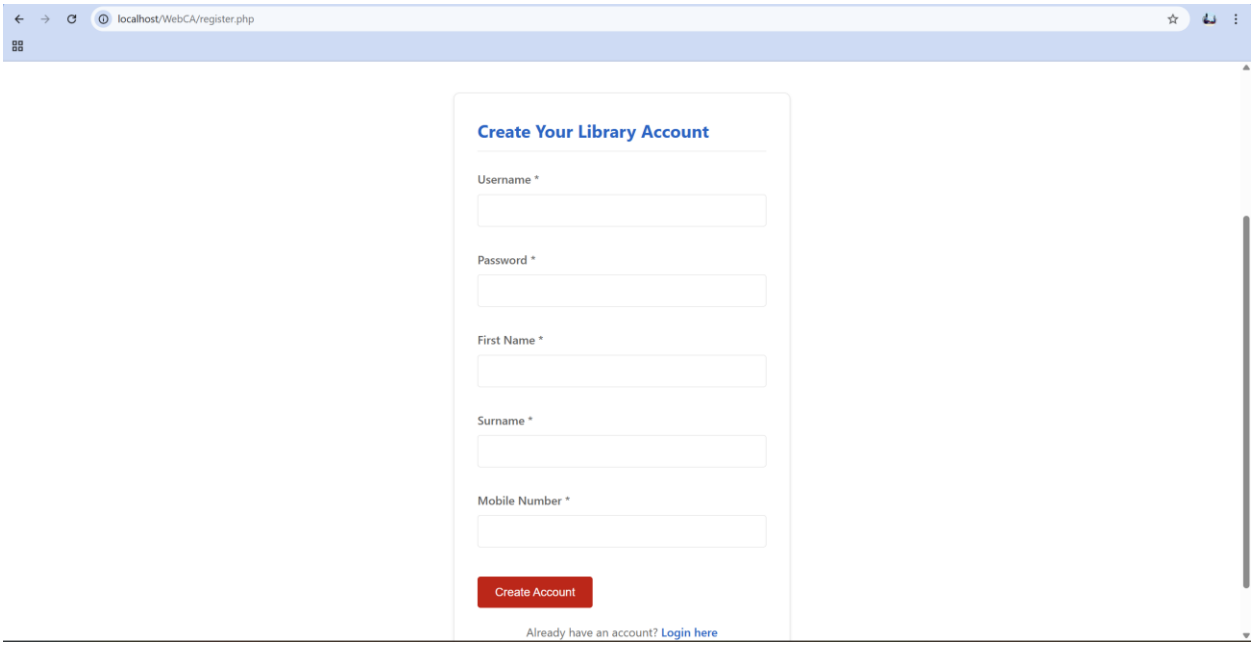
```
        }
    }
```

5. Screenshots

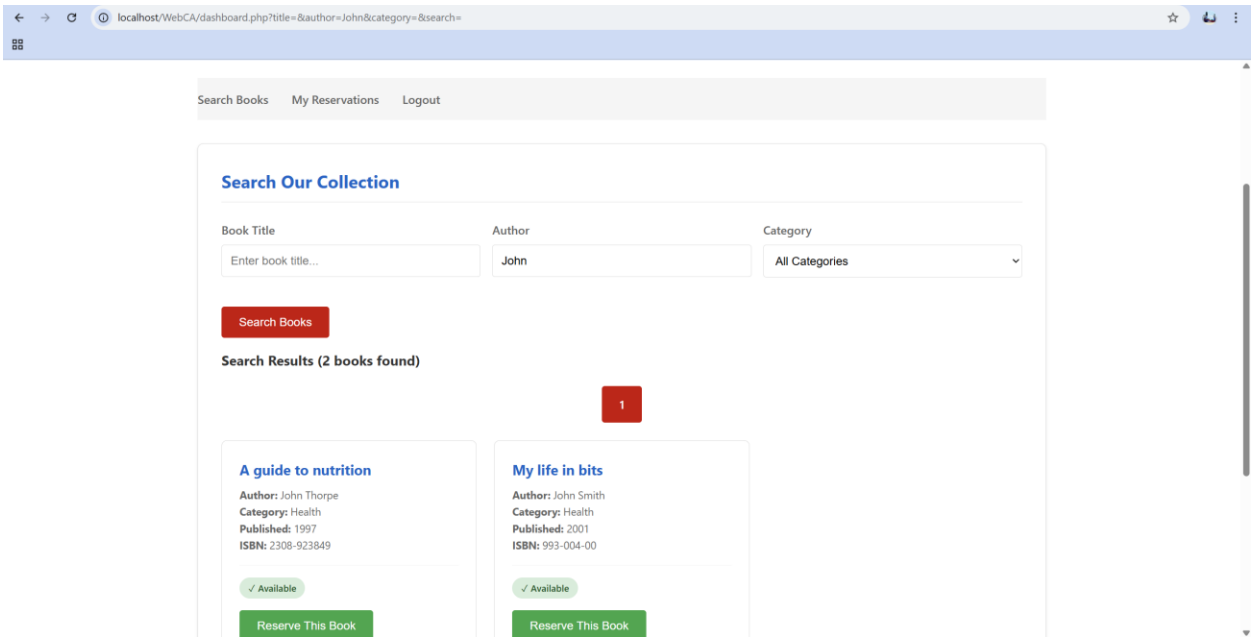
Screenshot 1: Login Page

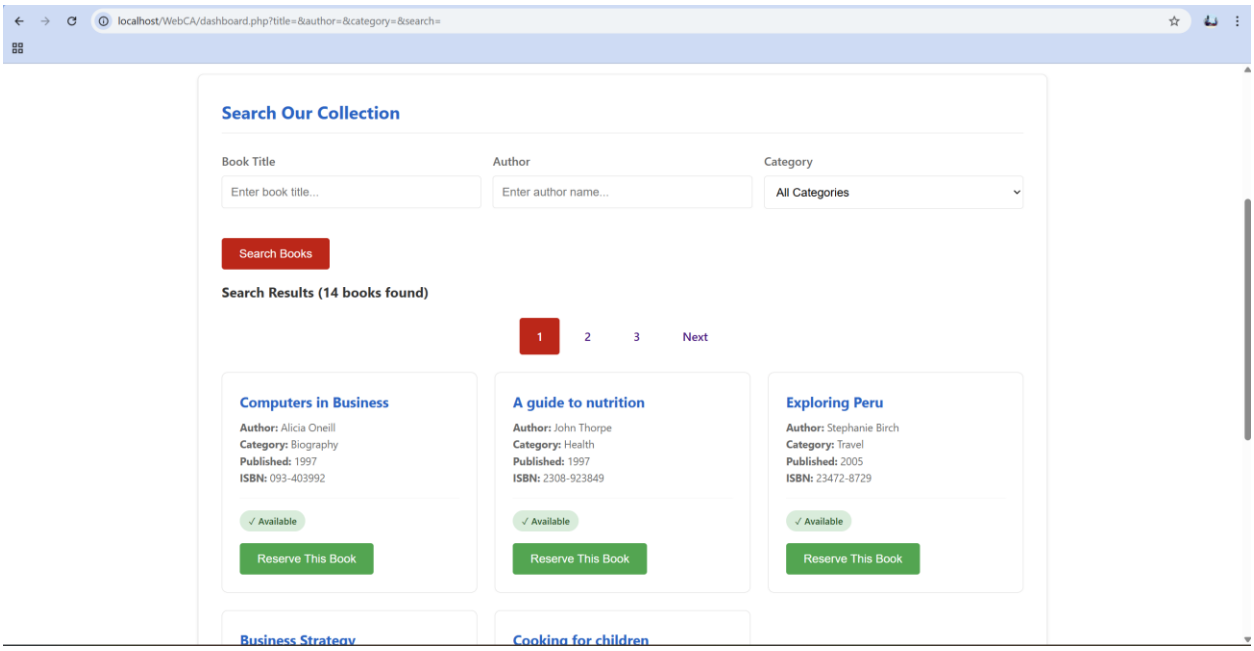


Screenshot 2: Register Page

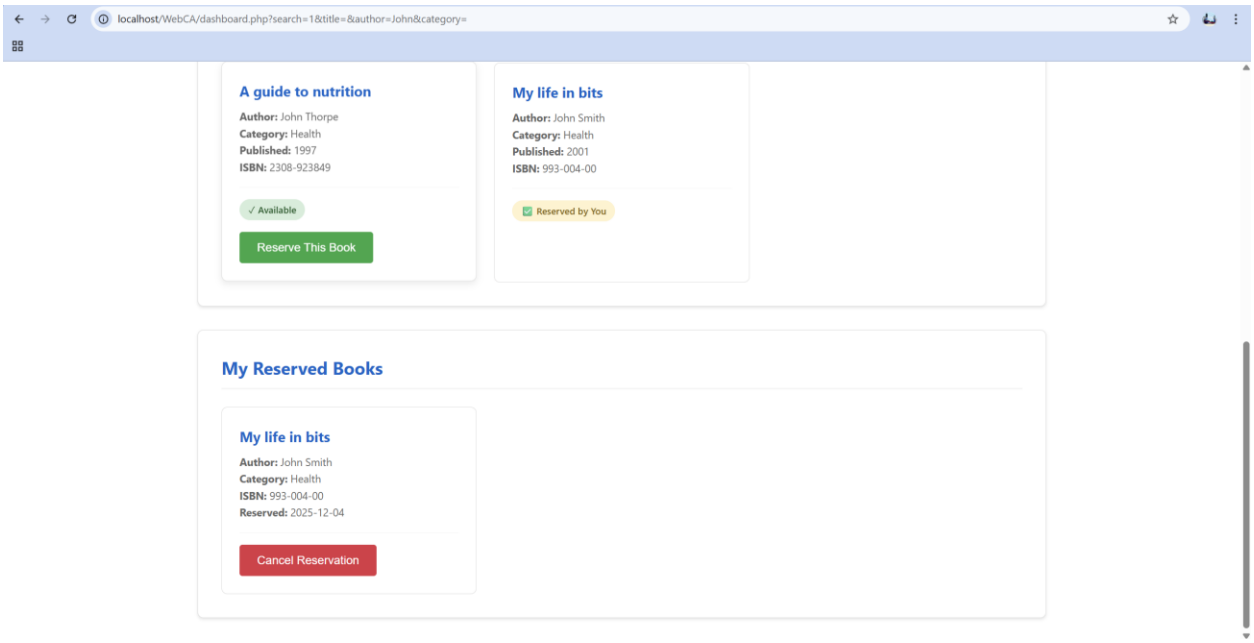


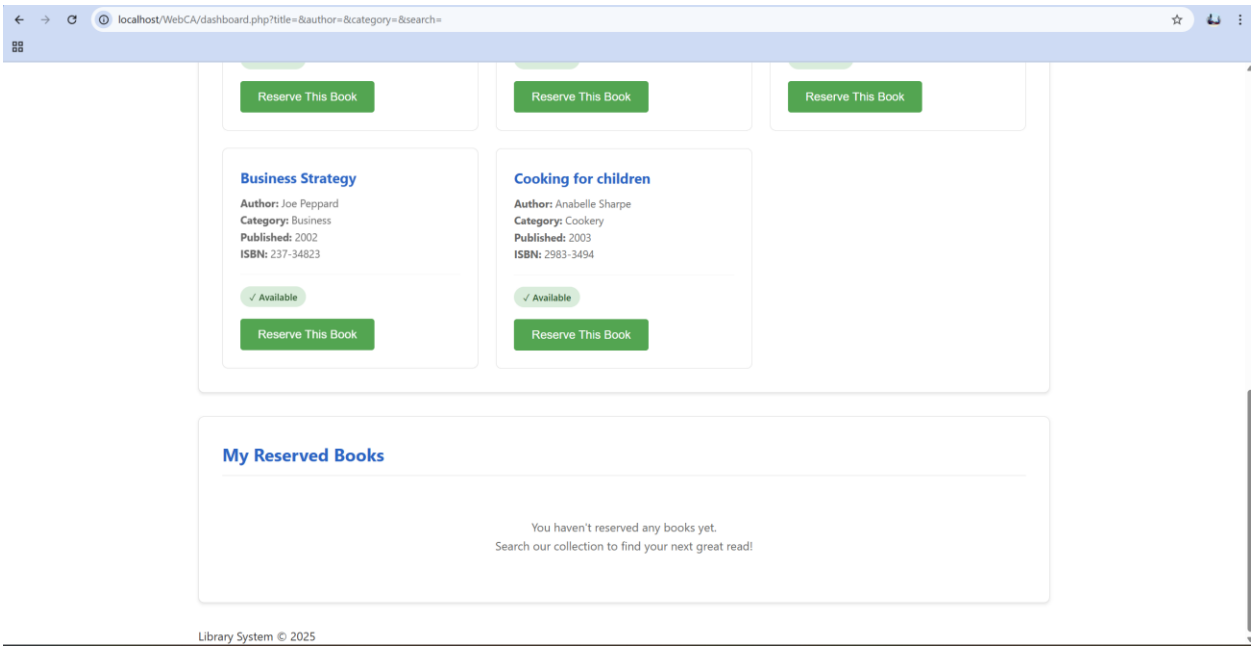
Screenshot 3,4: Search Books





Screenshot 5: My Reservations

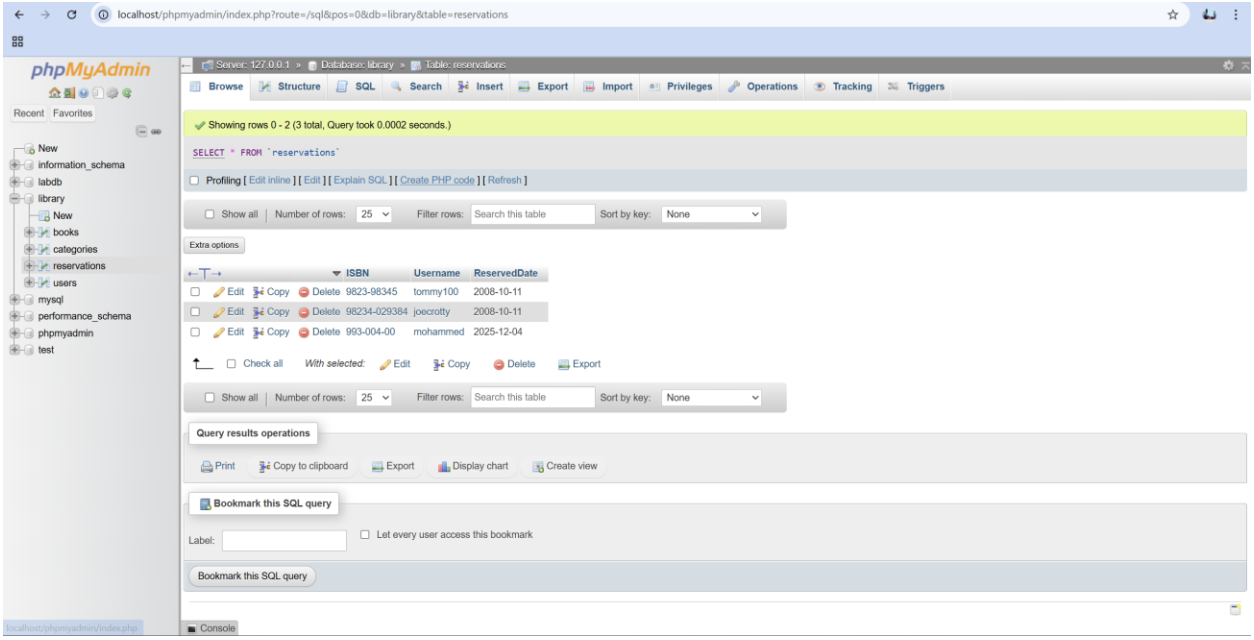


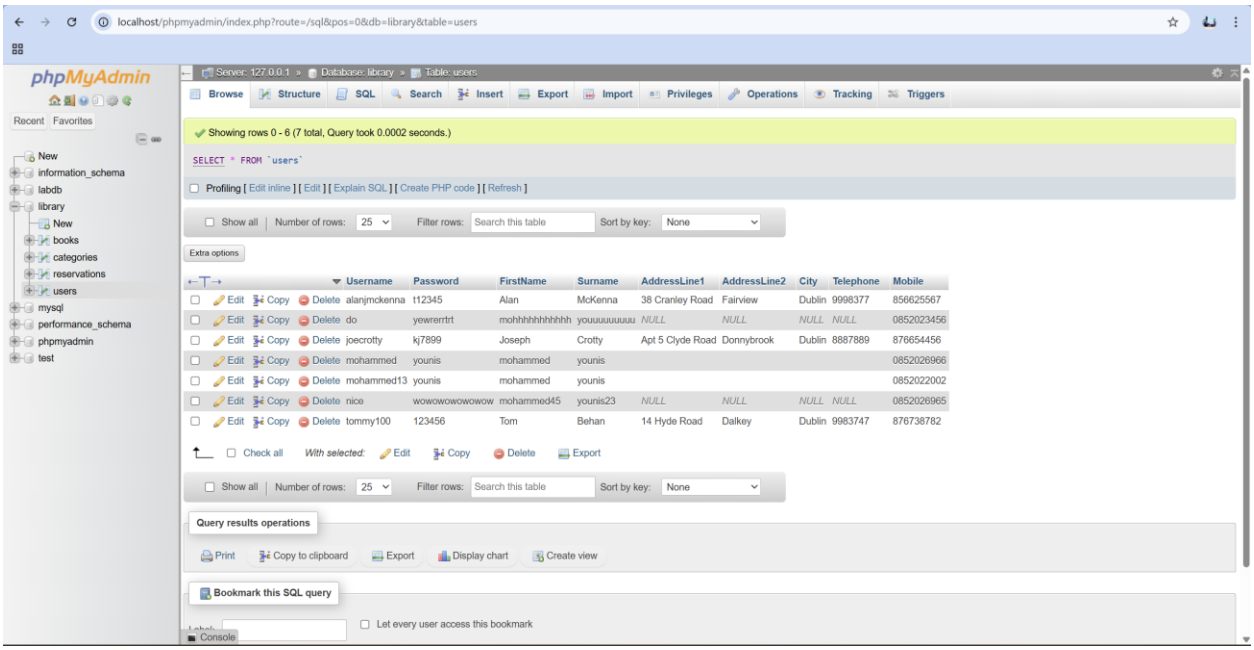


6. Database

There are 4 tables:

1. **Users** - stores user accounts
2. **Books** - stores all the books
3. **Categories** - stores book categories like Fiction, Business
4. **Reservations** - tracks who reserved which books





7. Testing Notes

I tested:

- Login with correct and wrong passwords
- Registration with valid and invalid mobile numbers
- Searching for books
- Reserving books
- Cancelling reservations

- Logging out and back in

Everything works as it should.